ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		В. В. Жукалин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

ФИЗИЧЕСКАЯ РЕАЛИЗАЦИЯ ЛОКАЛЬНОЙ БАЗЫ ДАННЫХ, ЗАПОЛНЕНИЕ БД И СЕРВЕРНАЯ АВТОРИЗАЦИЯ

по курсу: Web-программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		И.А. Лаврешин
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Спроектировать и физически создать локальную реляционную базу SignalBoard в PostgreSQL, объяснить назначение и содержимое каждой таблицы, заполнить базу начальными данными через Prisma Client и реализовать учебную серверную авторизацию на Next.js с хешированием паролей, подписанной cookie-сессией и проверкой прав перед изменением отслеживаемых сигналов.

2 Задание

Удалить временную модель SetupCheck из лабораторной работы № 5, описать модели User, NeighborSignal и TrackedSignal, выбрать физические типы PostgreSQL, назначить первичные, уникальные и внешние ключи, создать составной первичный ключ таблицы отслеживания, задать правила onDelete и индекс для обратного поиска по signalId. Сформировать первую миграцию Prisma Migrate, изучить сгенерированный SQL, проверить физические таблицы и ограничения. Написать идемпотентный seed-скрипт, реализовать серверную валидацию, хеширование пароля bcrypt, подписанную сессию jose в cookie с безопасными атрибутами, маршруты регистрации, входа, текущего пользователя и выхода, не возвращать passwordHash клиенту и защитить изменение отслеживания серверной проверкой пользователя. Проверить npm run lint и npm run build.

3 Ход выполнения задания

Временная модель SetupCheck удалена. Предметная область SignalBoard описана тремя таблицами. Схема связей: User 1 → N TrackedSignal N ← 1 NeighborSignal. Таблица TrackedSignal реализует явную связь «многие-ко-многим»: один пользователь отслеживает много сигналов, один сигнал отслеживают многие пользователи.

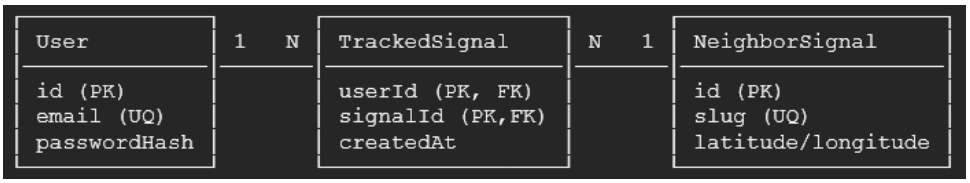


Рисунок 1 – Связи

Назначение таблиц:

– User – зарегистрированные соседи: имя, уникальный email и bcrypt-хеш пароля,

– NeighborSignal – локальные сигналы, которые раньше хранились в массиве app/data/signals.ts: заголовок, описание, тип, место, изображение и координаты,

– TrackedSignal – связи «пользователь отслеживает сигнал» с временем создания.

Таблица 1 – Столбцы таблицы User

Столбец	**Тип PostgreSQL**	**NULL**	**Ограничения**
id	UUID	NOT NULL	PRIMARY KEY, uuid() в Prisma
name	VARCHAR(80)	NOT NULL	Длина 2–80 проверяется сервером
email	VARCHAR(254)	NOT NULL	UNIQUE (User_email_key)
passwordHash	VARCHAR(255)	NOT NULL	Только bcrypt-хеш
createdAt	TIMESTAMPTZ(3)	NOT NULL	DEFAULT CURRENT_TIMESTAMP
updatedAt	TIMESTAMPTZ(3)	NOT NULL	@updatedAt

Таблица 2 – Столбцы таблицы NeighborSignal

Столбец	**Тип PostgreSQL**	**NULL**	**Ограничения**
id	UUID	NOT NULL	PRIMARY KEY
slug	VARCHAR(80)	NOT NULL	UNIQUE, читаемый идентификатор
title	VARCHAR(160)	NOT NULL	—
summary	TEXT	NOT NULL	—
category	VARCHAR(40)	NOT NULL	ищу / отдаю / предупреждаю
location	VARCHAR(240)	NOT NULL	—
image, imageAlt	VARCHAR(240)	NOT NULL	Путь и альтернативный текст
latitude, longitude	DECIMAL(9,6)	NOT NULL	Точные координаты без ошибок float
createdAt, updatedAt	TIMESTAMPTZ(3)	NOT NULL	Метки времени

Таблица 3 – Столбцы таблицы TrackedSignal

Столбец	**Тип PostgreSQL**	**NULL**	**Ограничения**
userId	UUID	NOT NULL	PK (часть), FK → User.id ON DELETE CASCADE
signalId	UUID	NOT NULL	PK (часть), FK → NeighborSignal.id ON DELETE CASCADE, INDEX
createdAt	TIMESTAMPTZ(3)	NOT NULL	DEFAULT CURRENT_TIMESTAMP

Составной первичный ключ (userId, signalId) одновременно идентифицирует связь и запрещает повтор одной пары: пользователь не может дважды отслеживать один сигнал. Отдельный суррогатный id был бы лишним, а уникальность пришлось бы обеспечивать дополнительным индексом. Индекс по первому столбцу ключа (userId) создаётся вместе с ключом, поэтому для обратного поиска «кто отслеживает сигнал» добавлен отдельный индекс TrackedSignal_signalId_idx.

onDelete: Cascade выбран потому, что строка TrackedSignal не имеет смысла без пользователя или сигнала: при удалении аккаунта или сигнала все его связи отслеживания удаляются автоматически, и в базе не остаётся «висячих» ссылок.

Файл prisma/schema.prisma:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id           String   @id @default(uuid()) @db.Uuid
  name         String   @db.VarChar(80)
  email        String   @unique @db.VarChar(254)
  passwordHash String   @db.VarChar(255)
  createdAt    DateTime @default(now()) @db.Timestamptz(3)
  updatedAt    DateTime @updatedAt @db.Timestamptz(3)

  tracked TrackedSignal[]
}

model NeighborSignal {
  id        String   @id @default(uuid()) @db.Uuid
  slug      String   @unique @db.VarChar(80)
  title     String   @db.VarChar(160)
  summary   String   @db.Text
  category  String   @db.VarChar(40)
  location  String   @db.VarChar(240)
  image     String   @db.VarChar(240)
  imageAlt  String   @db.VarChar(240)
  latitude  Decimal  @db.Decimal(9, 6)
  longitude Decimal  @db.Decimal(9, 6)
  createdAt DateTime @default(now()) @db.Timestamptz(3)
  updatedAt DateTime @updatedAt @db.Timestamptz(3)

  tracked TrackedSignal[]
}

model TrackedSignal {
  userId    String   @db.Uuid
  signalId  String   @db.Uuid
  createdAt DateTime @default(now()) @db.Timestamptz(3)

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  signal NeighborSignal @relation(fields: [signalId], references: [id],
onDelete: Cascade)

  @@id([userId, signalId])
  @@index([signalId])
}
```

Первая миграция init_signalboard сформирована командой npx prisma migrate dev --name init_signalboard и применена к базе signalboard. До заполнения базы изучен сгенерированный SQL: три CREATE TABLE с первичными ключами, два уникальных индекса (email, slug), индекс TrackedSignal_signalId_idx и два внешних ключа с ON DELETE CASCADE (рисунок 2):

```
CREATE TABLE "User" (
    "id" UUID NOT NULL,
    "name" VARCHAR(80) NOT NULL,
    "email" VARCHAR(254) NOT NULL,
    "passwordHash" VARCHAR(255) NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMPTZ(3) NOT NULL,

    CONSTRAINT "User_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "NeighborSignal" (
    "id" UUID NOT NULL,
    "slug" VARCHAR(80) NOT NULL,
    "title" VARCHAR(160) NOT NULL,
    "summary" TEXT NOT NULL,
    "category" VARCHAR(40) NOT NULL,
    "location" VARCHAR(240) NOT NULL,
    "image" VARCHAR(240) NOT NULL,
    "imageAlt" VARCHAR(240) NOT NULL,
    "latitude" DECIMAL(9,6) NOT NULL,
    "longitude" DECIMAL(9,6) NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMPTZ(3) NOT NULL,

    CONSTRAINT "NeighborSignal_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "TrackedSignal" (
    "userId" UUID NOT NULL,
    "signalId" UUID NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT "TrackedSignal_pkey" PRIMARY KEY ("userId","signalId")
);

CREATE UNIQUE INDEX "User_email_key" ON "User"("email");

CREATE UNIQUE INDEX "NeighborSignal_slug_key" ON "NeighborSignal"("slug");

CREATE INDEX "TrackedSignal_signalId_idx" ON "TrackedSignal"("signalId");

ALTER TABLE "TrackedSignal" ADD CONSTRAINT "TrackedSignal_userId_fkey" FOREIGN KEY
("userId") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE "TrackedSignal" ADD CONSTRAINT "TrackedSignal_signalId_fkey" FOREIGN KEY
("signalId") REFERENCES "NeighborSignal"("id") ON DELETE CASCADE ON UPDATE CASCADE;
```

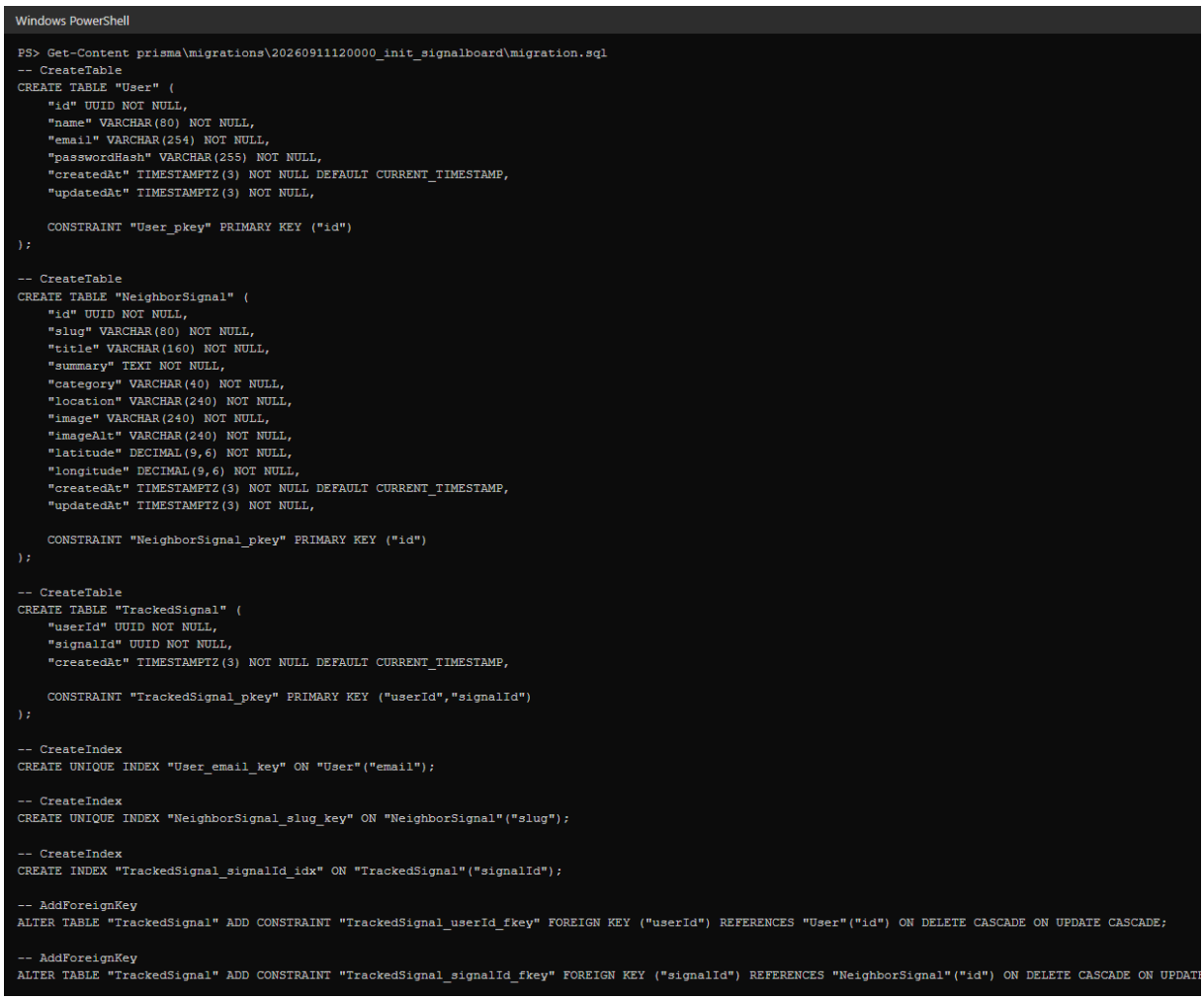


Рисунок 2 – Файл migration.sql первой миграции

Команда npx prisma migrate status (рисунок 3) сообщает, что найдена одна миграция и схема базы актуальна. Физические таблицы проверены в psql под ролью signal_app метакомандой \d (рисунки 4–6): видны типы столбцов, NOT NULL, значения по умолчанию, первичные ключи, уникальные индексы, составной ключ и внешние ключи TrackedSignal.

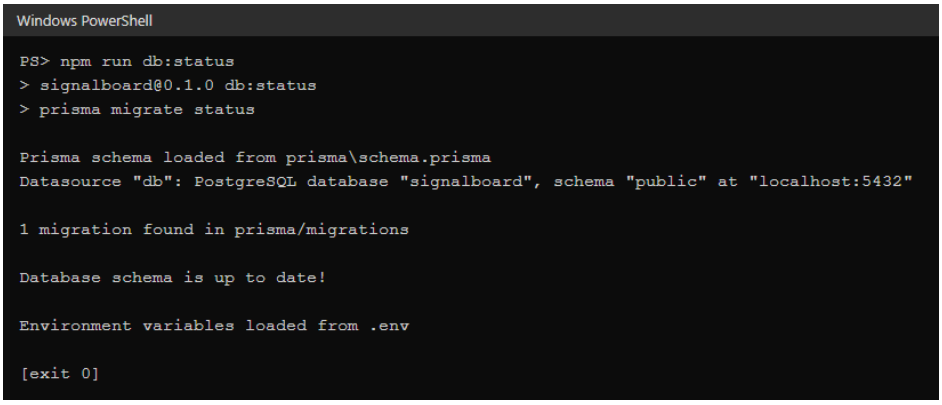


Рисунок 3 – Вывод npx prisma migrate status

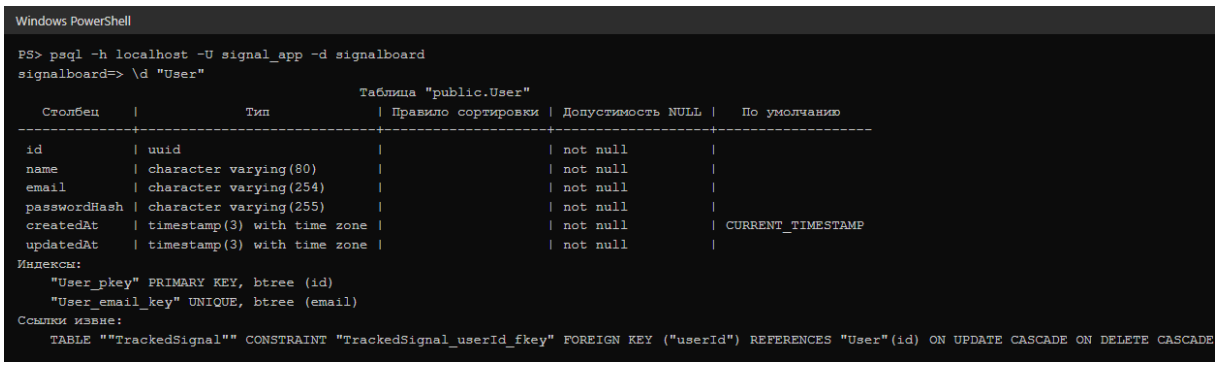


Рисунок 4 – Физическая таблица User (\d)

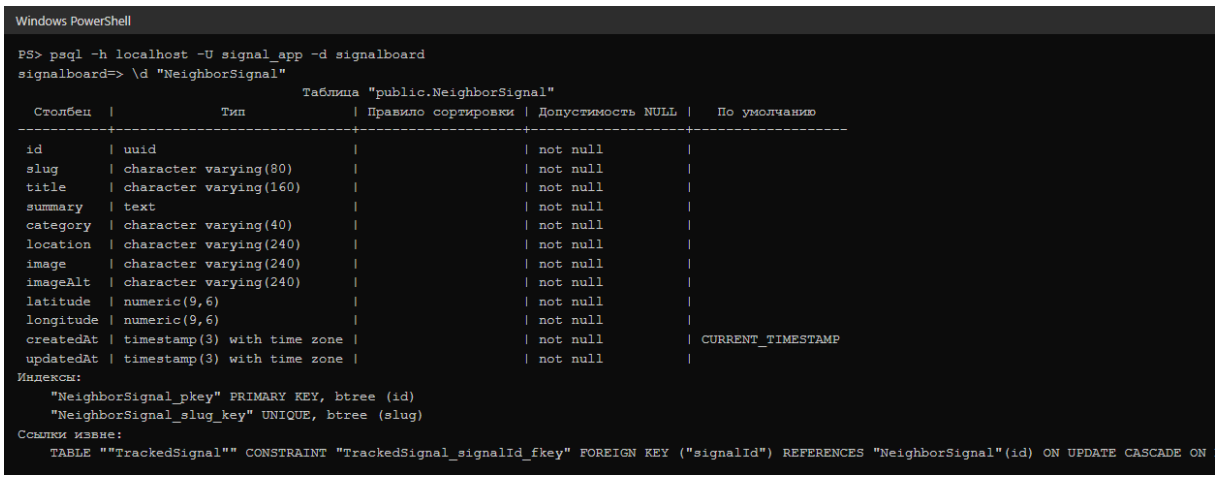


Рисунок 5 – Физическая таблица NeighborSignal (\d)

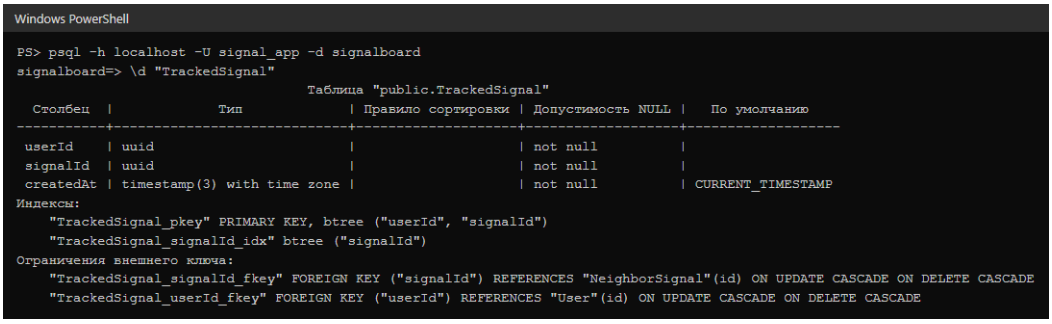


Рисунок 6 – Физическая таблица TrackedSignal (\d)

Seed-скрипт prisma/seed.ts идемпотентен: пользователь создаётся через upsert по email, каждый сигнал – через upsert по slug, связь отслеживания – через upsert по составному ключу userId_signalId. Пароль демонстрационного пользователя хешируется bcrypt с cost 12, в консоль выводится только итог без пароля и хеша, PrismaClient отключается в finally. Сигналы совпадают с данными ЛР3: три сигнала в доме на Плесецкой улице, 16, с фотографиями из public/images/places.

Seed запущен дважды (рисунок 7). После обоих запусков count(*) равен 1 для User, 3 для NeighborSignal и 1 для TrackedSignal, то есть повторный запуск не увеличил число строк. Таблицы в Prisma Studio показаны на рисунках 8–10: в User хранится только хеш вида $2b$12$..., в TrackedSignal видны связи user и signal.

```
Windows PowerShell

PS> npm run db:seed
> signalboard@0.1.0 db:seed
> prisma db seed

Running seed command `tsx prisma/seed.ts` ...
Seed complete: 1 user, 3 signals, 1 tracked

The seed command has been executed.

Environment variables loaded from .env

[exit 0]

PS> npm run db:seed
> signalboard@0.1.0 db:seed
> prisma db seed

Running seed command `tsx prisma/seed.ts` ...
Seed complete: 1 user, 3 signals, 1 tracked

The seed command has been executed.

Environment variables loaded from .env

[exit 0]

PS> psql ... -c "SELECT ... count(*) ..."
   table_name    | count
-----------------+-------
 User            |     1
 NeighborSignal  |     3
 TrackedSignal   |     1
(3 строки)
```

Рисунок 7 – Два запуска seed и count(*) по трём таблицам

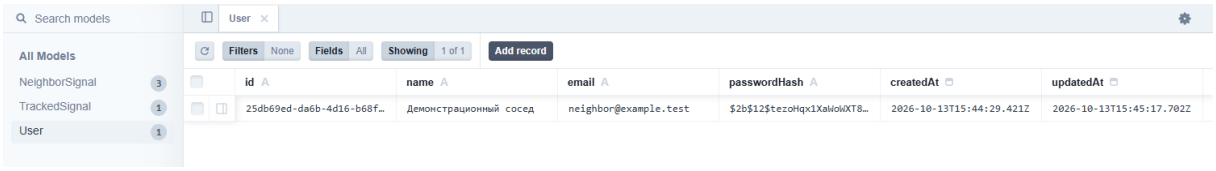


Рисунок 8 – Таблица User в Prisma Studio

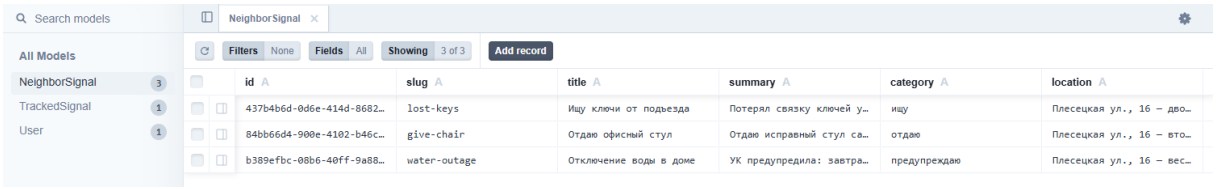


Рисунок 9 – Таблица NeighborSignal в Prisma Studio

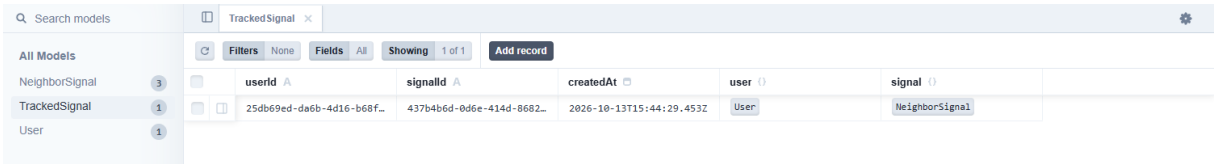


Рисунок 10 – Таблица TrackedSignal в Prisma Studio

Серверная авторизация состоит из нескольких модулей. app/lib/prisma.ts из ЛР5 остаётся единственным экземпляром PrismaClient. app/lib/validation.ts нормализует email (trim и нижний регистр) и проверяет имя (2–80 символов после trim), формат email, длину пароля (не менее 8) и совпадение подтверждения, любые входные значения приводятся к строкам, а не принимаются как готовые данные. app/lib/auth.ts подписывает JWT алгоритмом HS256 секретом AUTH_SECRET через библиотеку jose (signSession), устанавливает cookie (createSession), проверяет подпись и срок (readSession) и удаляет cookie (deleteSession). Payload содержит только userId и name, срок – 7 дней. Cookie signal_session имеет атрибуты httpOnly, sameSite=lax, path=/, maxAge 604800 секунд и secure в production. app/lib/current-user.ts после проверки токена загружает пользователя из базы и возвращает только id, name и email.

Маршруты API:

– POST /api/auth/register – валидация, bcrypt.hash(password, 12), создание User, сессия, 201, при ошибках полей 400, при занятом email 409,

– POST /api/auth/login – поиск по email, bcrypt.compare, 200, при пустых полях 400, при неверном email или пароле одно сообщение и 401,

– GET /api/auth/me – безопасные поля пользователя или user: null для гостя,

– POST /api/auth/logout – удаление cookie,

– POST и DELETE /api/favorites/[signalId] – сначала getCurrentUser, гость получает 401, POST проверяет существование сигнала (иначе 404) и выполняет upsert связи, DELETE удаляет связь только по user.id из сессии.

Сценарии проверены запросами к production-серверу (npm run start) с реальной базой (рисунки 11–12). Для проверки зарегистрирован отдельный пользователь, после проверки он удалён, а его связи удалены каскадом. Значение cookie на рисунках закрыто. Итог приведён в таблице 4.

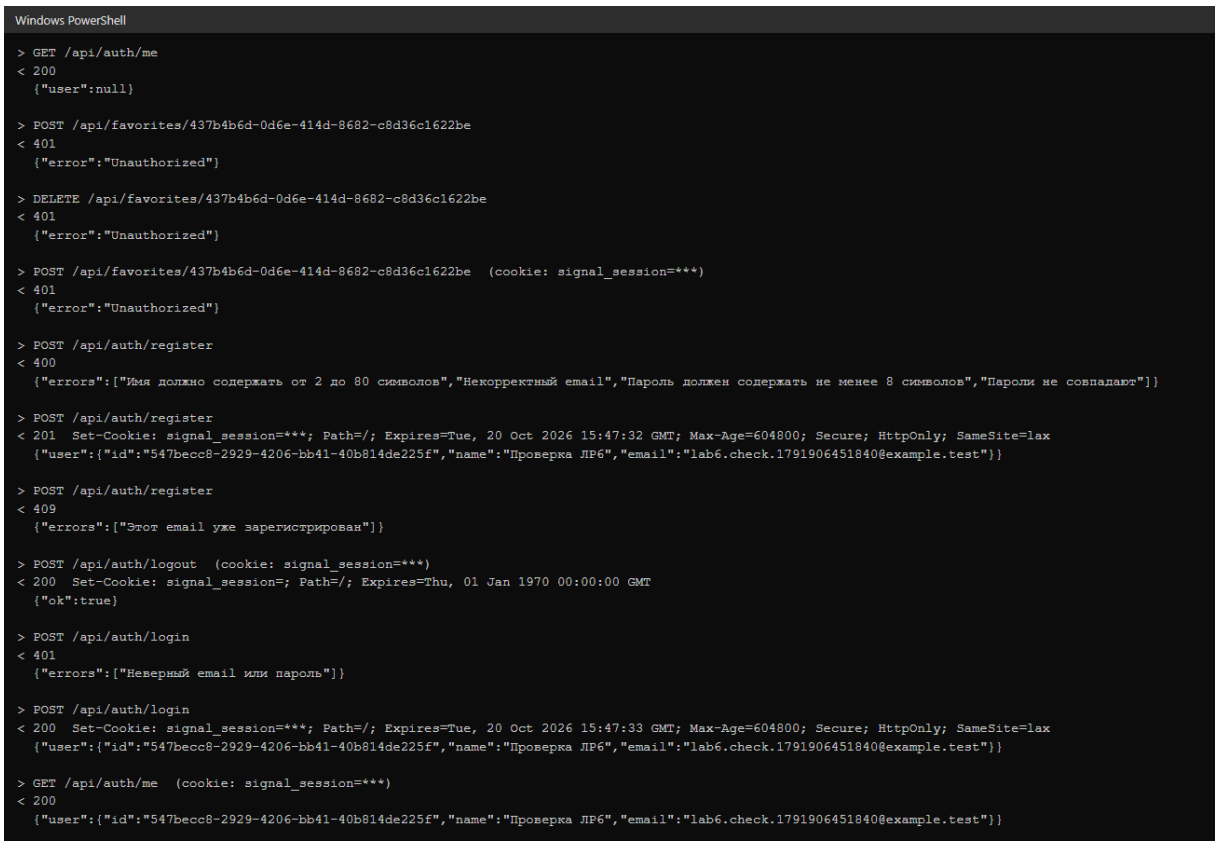


Рисунок 11 – HTTP-сценарии регистрации, входа, me и выхода

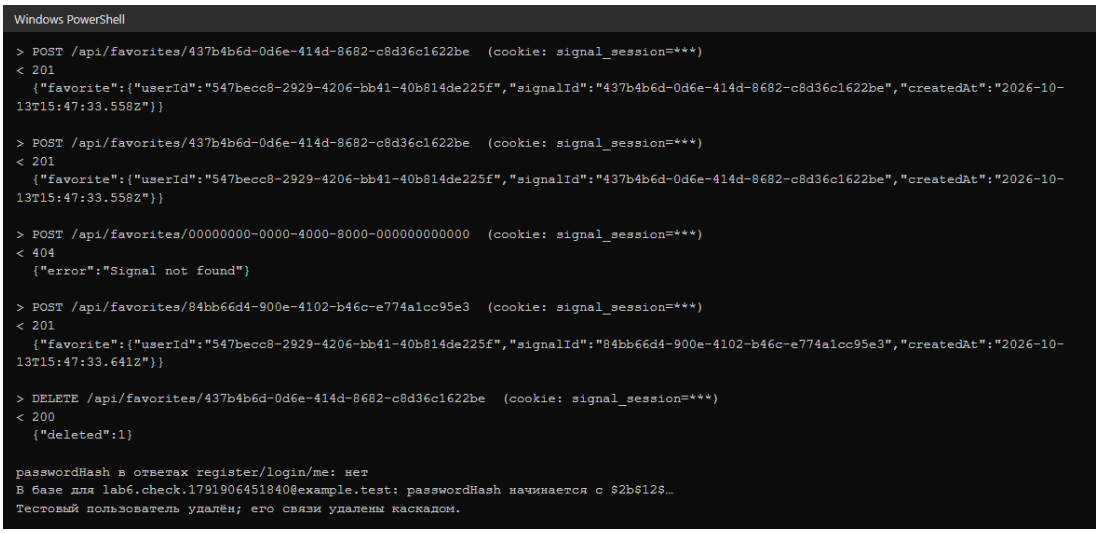


Рисунок 12 – HTTP-сценарии защищённого отслеживания

Таблица 4 – Матрица HTTP-сценариев

Сценарий	**Запрос**	**Ожидается**	**Получено**
Гость читает текущего пользователя	GET /api/auth/me	200, user: null	200, user: null
Гость добавляет отслеживание	POST /api/favorites/{id}	401	401

Сценарий	Запрос	Ожидается	Получено
Гость удаляет отслеживание	DELETE /api/favorites/{id}	401	401
Поддельная cookie сессии	POST /api/favorites/{id}	401	401
Некорректные поля регистрации	POST /api/auth/register	400	400
Регистрация нового email	POST /api/auth/register	201	201, Set-Cookie
Повторная регистрация email	POST /api/auth/register	409	409
Выход	POST /api/auth/logout	200	200, cookie удалена
Неверный пароль	POST /api/auth/login	401	401
Правильный вход	POST /api/auth/login	200 и Set-Cookie	200, Set-Cookie
Текущий пользователь после входа	GET /api/auth/me	объект user	200, id, name, email
Отслеживание после входа	POST /api/favorites/{id}	201	201
Повторное отслеживание	POST /api/favorites/{id}	без дубликата	201, строк не прибавилось
Несуществующий сигнал	POST /api/favorites/{uuid}	404	404
Удаление отслеживания	DELETE /api/favorites/{id}	только своя связь	200, deleted: 1; связь seed-пользователя цела

Отсутствие passwordHash в ответах обеспечено явным выбором полей: в register используется select { id, name, email }, в login формируется DTO из трёх полей, в current-user – select без passwordHash. Ответы register, login и me на рисунке 10 содержат только id, name и email, проверка тела всех трёх ответов на подстроку passwordHash дала отрицательный результат, а в базе для созданного пользователя хранится хеш, начинающийся с $2b$12$.

Итоговая проверка: npx prisma format и npx prisma validate (рисунок 13), npx prisma migrate status (рисунок 3), npx prisma generate, npm run lint и npm run build (рисунки 14–15) выполнены без ошибок.

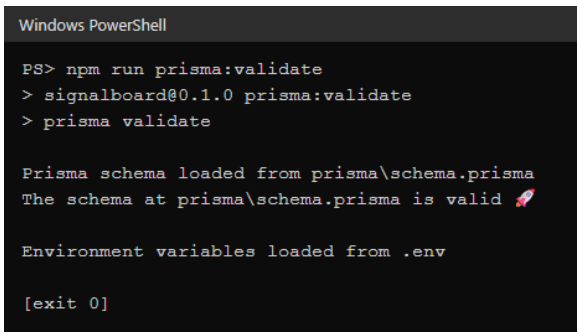


Рисунок 13 – Схема проходит prisma validate

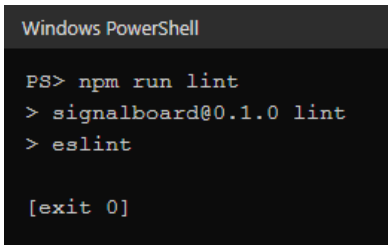


Рисунок 14 – Результат npm run lint

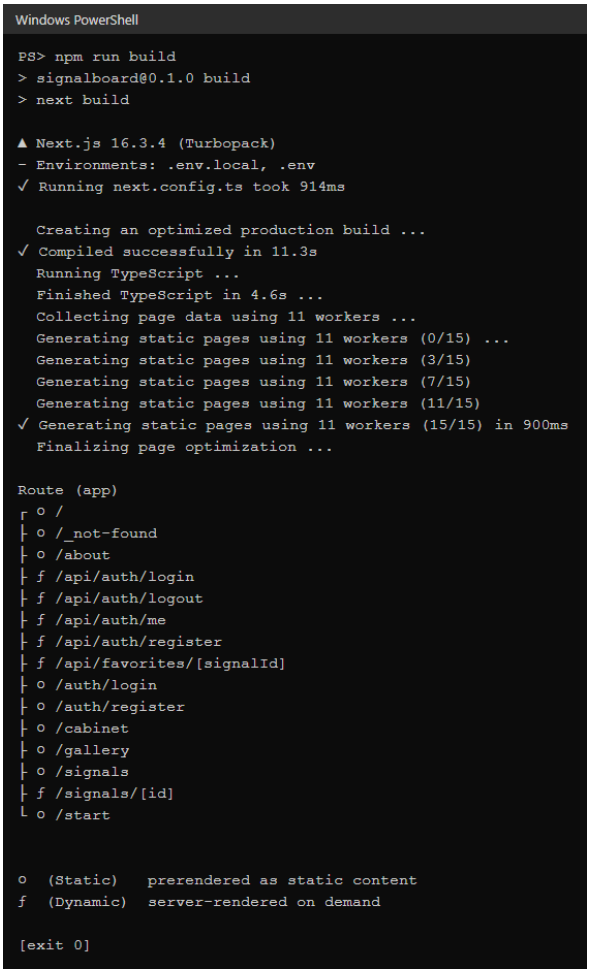


Рисунок 15 – Результат npm run build

4 Вывод

В ходе работы спроектирована физическая схема SignalBoard из таблиц User, NeighborSignal и TrackedSignal с UUID-ключами, уникальными email и slug, составным первичным ключом связи, внешними ключами ON DELETE CASCADE и индексом по signalId. Схема зафиксирована первой миграцией Prisma, база заполняется идемпотентным seed. Реализована серверная авторизация: валидация полей, bcrypt-хеширование с cost 12, подписанная jose-сессия в httpOnly-cookie, маршруты register, login, me и logout, а изменение отслеживания проверяет пользователя на сервере. Миграция применена, migrate status подтверждает актуальность схемы, два запуска seed дают 1, 3 и 1 строку. Все 15 HTTP-сценариев на реальной базе вернули ожидаемые статусы: регистрация 201, повторная регистрация 409, неверный пароль 401, вход 200, гость и поддельная сессия 401, отслеживание 201 без дубликатов, несуществующий сигнал 404, passwordHash не возвращается клиенту. Проект проходит npm run lint и npm run build.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

Файл prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id           String   @id @default(uuid()) @db.Uuid
  name         String   @db.VarChar(80)
  email        String   @unique @db.VarChar(254)
  passwordHash String   @db.VarChar(255)
  createdAt    DateTime @default(now()) @db.Timestamptz(3)
  updatedAt    DateTime @updatedAt @db.Timestamptz(3)

  tracked TrackedSignal[]
}

model NeighborSignal {
  id        String   @id @default(uuid()) @db.Uuid
  slug      String   @unique @db.VarChar(80)
  title     String   @db.VarChar(160)
  summary   String   @db.Text
  category  String   @db.VarChar(40)
  location  String   @db.VarChar(240)
  image     String   @db.VarChar(240)
  imageAlt  String   @db.VarChar(240)
  latitude  Decimal  @db.Decimal(9, 6)
  longitude Decimal  @db.Decimal(9, 6)
  createdAt DateTime @default(now()) @db.Timestamptz(3)
  updatedAt DateTime @updatedAt @db.Timestamptz(3)

  tracked TrackedSignal[]
}

model TrackedSignal {
  userId    String   @db.Uuid
  signalId  String   @db.Uuid
  createdAt DateTime @default(now()) @db.Timestamptz(3)

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  signal NeighborSignal @relation(fields: [signalId], references: [id],
onDelete: Cascade)

  @@id([userId, signalId])
  @@index([signalId])
}
```

Файл prisma/migrations/20260911120000_init_signalboard/migration.sql

```
-- CreateTable
CREATE TABLE "User" (
```

```
    "id" UUID NOT NULL,
    "name" VARCHAR(80) NOT NULL,
    "email" VARCHAR(254) NOT NULL,
    "passwordHash" VARCHAR(255) NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMPTZ(3) NOT NULL,

    CONSTRAINT "User_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "NeighborSignal" (
    "id" UUID NOT NULL,
    "slug" VARCHAR(80) NOT NULL,
    "title" VARCHAR(160) NOT NULL,
    "summary" TEXT NOT NULL,
    "category" VARCHAR(40) NOT NULL,
    "location" VARCHAR(240) NOT NULL,
    "image" VARCHAR(240) NOT NULL,
    "imageAlt" VARCHAR(240) NOT NULL,
    "latitude" DECIMAL(9,6) NOT NULL,
    "longitude" DECIMAL(9,6) NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMPTZ(3) NOT NULL,

    CONSTRAINT "NeighborSignal_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "TrackedSignal" (
    "userId" UUID NOT NULL,
    "signalId" UUID NOT NULL,
    "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT "TrackedSignal_pkey" PRIMARY KEY ("userId","signalId")
);

-- CreateIndex
CREATE UNIQUE INDEX "User_email_key" ON "User"("email");

-- CreateIndex
CREATE UNIQUE INDEX "NeighborSignal_slug_key" ON "NeighborSignal"("slug");

-- CreateIndex
CREATE INDEX "TrackedSignal_signalId_idx" ON "TrackedSignal"("signalId");

-- AddForeignKey
ALTER TABLE "TrackedSignal" ADD CONSTRAINT "TrackedSignal_userId_fkey"
FOREIGN KEY ("userId") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE
CASCADE;

-- AddForeignKey
ALTER TABLE "TrackedSignal" ADD CONSTRAINT "TrackedSignal_signalId_fkey"
FOREIGN KEY ("signalId") REFERENCES "NeighborSignal"("id") ON DELETE CASCADE
ON UPDATE CASCADE;
```

Файл prisma/seed.ts

```
import "dotenv/config";
import { PrismaClient } from "@prisma/client";
import { hash } from "bcrypt";
```

```
const prisma = new PrismaClient();

const signals = [
  {
    slug: "lost-keys",
    title: "Ищу ключи от подъезда",
    summary:
      "Потерял связку ключей у детской площадки во дворе дома на Плесецкой 16
вечером, нужна помощь соседей.",
    category: "ищу",
    location: "Плесецкая ул., 16 — двор, детская площадка",
    image: "/images/places/lost-keys.jpg",
    imageAlt: "Детская площадка во дворе дома на Плесецкой 16",
    latitude: "60.036100",
    longitude: "30.221450",
  },
  {
    slug: "give-chair",
    title: "Отдаю офисный стул",
    summary:
      "Отдаю исправный стул самовывозом из второго подъезда дома на Плесецкой
16, забирать сегодня-завтра.",
    category: "отдаю",
    location: "Плесецкая ул., 16 — второй подъезд, 1 этаж",
    image: "/images/places/give-chair.jpg",
    imageAlt: "Офисный стул у входа во второй подъезд дома на Плесецкой 16",
    latitude: "60.035820",
    longitude: "30.222050",
  },
  {
    slug: "water-outage",
    title: "Отключение воды в доме",
    summary:
      "УК предупредила: завтра с 10:00 до 16:00 не будет холодной воды по
всему стояку в доме на Плесецкой 16.",
    category: "предупреждаю",
    location: "Плесецкая ул., 16 — весь дом, стояк ХВС",
    image: "/images/places/water-outage.jpg",
    imageAlt: "Технический стояк холодной воды в подъезде дома на Плесецкой
16",
    latitude: "60.035620",
    longitude: "30.221350",
  },
];

async function main() {
  const passwordHash = await hash("12345678", 12);

  const user = await prisma.user.upsert({
    where: { email: "neighbor@example.test" },
    update: {
      name: "Демонстрационный сосед",
      passwordHash,
    },
    create: {
      name: "Демонстрационный сосед",
      email: "neighbor@example.test",
      passwordHash,
    },
  });
```

```
  for (const signal of signals) {
    await prisma.neighborSignal.upsert({
      where: { slug: signal.slug },
      update: signal,
      create: signal,
    });
  }

  const lostKeys = await prisma.neighborSignal.findUniqueOrThrow({
    where: { slug: "lost-keys" },
  });

  await prisma.trackedSignal.upsert({
    where: {
      userId_signalId: {
        userId: user.id,
        signalId: lostKeys.id,
      },
    },
    update: {},
    create: {
      userId: user.id,
      signalId: lostKeys.id,
    },
  });

  console.log("Seed complete: 1 user, 3 signals, 1 tracked");
}

main()
  .catch((error) => {
    console.error("Seed failed:", error);
    process.exitCode = 1;
  })
  .finally(async () => {
    await prisma.$disconnect();
  });
```

Файл app/lib/validation.ts

```
export type RegistrationInput = {
  name: string;
  email: string;
  password: string;
  confirmPassword: string;
};

export function normalizeEmail(value: unknown): string {
  return String(value ?? "").trim().toLowerCase();
}

export function validateRegistration(value: unknown): {
  data: RegistrationInput | null;
  errors: string[];
} {
  const source =
    typeof value === "object" && value !== null
      ? (value as Record<string, unknown>)
      : {};

  const data: RegistrationInput = {
    name: String(source.name ?? "").trim(),
```

```
    email: normalizeEmail(source.email),
    password: String(source.password ?? ""),
    confirmPassword: String(source.confirmPassword ?? ""),
  };

  const errors: string[] = [];

  if (data.name.length < 2 || data.name.length > 80) {
    errors.push("Имя должно содержать от 2 до 80 символов");
  }

  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(data.email)) {
    errors.push("Некорректный email");
  }

  if (data.password.length < 8) {
    errors.push("Пароль должен содержать не менее 8 символов");
  }

  if (data.password !== data.confirmPassword) {
    errors.push("Пароли не совпадают");
  }

  return {
    data: errors.length === 0 ? data : null,
    errors,
  };
}
```

Файл app/lib/auth.ts

```
import { SignJWT, jwtVerify } from "jose";
import { cookies } from "next/headers";

const COOKIE_NAME = "signal_session";
const SESSION_MAX_AGE = 60 * 60 * 24 * 7;

function getSecret(): Uint8Array {
  const value = process.env.AUTH_SECRET;
  if (!value) {
    throw new Error("AUTH_SECRET is not configured");
  }
  return new TextEncoder().encode(value);
}

export type SessionPayload = {
  userId: string;
  name: string;
};

async function signSession(payload: SessionPayload): Promise<string> {
  return new SignJWT(payload)
    .setProtectedHeader({ alg: "HS256" })
    .setIssuedAt()
    .setExpirationTime("7d")
    .sign(getSecret());
}

export async function createSession(payload: SessionPayload): Promise<void> {
  const token = await signSession(payload);
  const store = await cookies();
```

```
  store.set(COOKIE_NAME, token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax",
    path: "/",
    maxAge: SESSION_MAX_AGE,
  });
}

export async function readSession(): Promise<SessionPayload | null> {
  const token = (await cookies()).get(COOKIE_NAME)?.value;
  if (!token) return null;

  try {
    const { payload } = await jwtVerify(token, getSecret(), {
      algorithms: ["HS256"],
    });

    if (typeof payload.userId !== "string" || typeof payload.name !==
"string") {
      return null;
    }

    return { userId: payload.userId, name: payload.name };
  } catch {
    return null;
  }
}

export async function deleteSession(): Promise<void> {
  (await cookies()).delete(COOKIE_NAME);
}
```

Файл app/lib/current-user.ts

```
import { prisma } from "./prisma";
import { readSession } from "./auth";

export async function getCurrentUser() {
  const session = await readSession();
  if (!session) return null;

  return prisma.user.findUnique({
    where: { id: session.userId },
    select: {
      id: true,
      name: true,
      email: true,
    },
  });
}
```

Файл app/api/auth/register/route.ts

```
import { Prisma } from "@prisma/client";
import { hash } from "bcrypt";
import { NextResponse } from "next/server";
import { createSession } from "@/app/lib/auth";
import { prisma } from "@/app/lib/prisma";
import { validateRegistration } from "@/app/lib/validation";

export const runtime = "nodejs";
```

```
export async function POST(request: Request) {
  try {
    const body = await request.json();
    const validation = validateRegistration(body);

    if (!validation.data) {
      return NextResponse.json(
        { errors: validation.errors },
        { status: 400 },
      );
    }

    const { name, email, password } = validation.data;

    const passwordHash = await hash(password, 12);

    const user = await prisma.user.create({
      data: { name, email, passwordHash },
      select: { id: true, name: true, email: true },
    });

    await createSession({ userId: user.id, name: user.name });

    return NextResponse.json({ user }, { status: 201 });
  } catch (error) {
    if (
      error instanceof Prisma.PrismaClientKnownRequestError &&
      error.code === "P2002"
    ) {
      return NextResponse.json(
        { errors: ["Этот email уже зарегистрирован"] },
        { status: 409 },
      );
    }

    return NextResponse.json(
      { errors: ["Внутренняя ошибка сервера"] },
      { status: 500 },
    );
  }
}
```

Файл app/api/auth/login/route.ts

```
import { compare } from "bcrypt";
import { NextResponse } from "next/server";
import { createSession } from "@/app/lib/auth";
import { prisma } from "@/app/lib/prisma";
import { normalizeEmail } from "@/app/lib/validation";

export const runtime = "nodejs";

export async function POST(request: Request) {
  try {
    const body = (await request.json()) as Record<string, unknown>;
    const email = normalizeEmail(body.email);
    const password = String(body.password ?? "");

    if (!email || !password) {
      return NextResponse.json(
```

```
        { errors: ["Заполните email и пароль"] },
        { status: 400 },
      );
    }

    const account = await prisma.user.findUnique({ where: { email } });

    if (!account || !(await compare(password, account.passwordHash))) {
      return NextResponse.json(
        { errors: ["Неверный email или пароль"] },
        { status: 401 },
      );
    }

    await createSession({ userId: account.id, name: account.name });

    const user = {
      id: account.id,
      name: account.name,
      email: account.email,
    };

    return NextResponse.json({ user });
  } catch {
    return NextResponse.json(
      { errors: ["Внутренняя ошибка сервера"] },
      { status: 500 },
    );
  }
}
```

Файл app/api/auth/me/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser } from "@/app/lib/current-user";

export async function GET() {
  const user = await getCurrentUser();
  return NextResponse.json({ user });
}
```

Файл app/api/auth/logout/route.ts

```
import { NextResponse } from "next/server";
import { deleteSession } from "@/app/lib/auth";

export async function POST() {
  await deleteSession();
  return NextResponse.json({ ok: true });
}
```

Файл app/api/favorites/[signalId]/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser } from "@/app/lib/current-user";
import { prisma } from "@/app/lib/prisma";

type RouteParams = {
  params: Promise<{ signalId: string }>;
};

export async function POST(_request: Request, context: RouteParams) {
  const user = await getCurrentUser();
```

```
  if (!user) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  const { signalId } = await context.params;

  const signal = await prisma.neighborSignal.findUnique({
    where: { id: signalId },
    select: { id: true },
  });
  if (!signal) {
    return NextResponse.json({ error: "Signal not found" }, { status: 404 });
  }

  const favorite = await prisma.trackedSignal.upsert({
    where: { userId_signalId: { userId: user.id, signalId } },
    update: {},
    create: { userId: user.id, signalId },
  });

  return NextResponse.json({ favorite }, { status: 201 });
}

export async function DELETE(_request: Request, context: RouteParams) {
  const user = await getCurrentUser();
  if (!user) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  const { signalId } = await context.params;

  const result = await prisma.trackedSignal.deleteMany({
    where: { userId: user.id, signalId },
  });

  return NextResponse.json({ deleted: result.count });
}
```